

Abstract

As a computer engineering student, we are taught from 8085 and 8086 architecture, assembly programming to high level object oriented programming language. Different subjects such as Microprocessors, Computer Organization and Architecture, Data Structure and Algorithm and Operating System that are part of our Engineering curriculum gives sufficient knowledge to build our own simplest operating system. This will provide us core knowledge and implementation of our curriculum as an Engineer. In order to implement these knowledge, developing a basic operating system from scratch would be the best way to do so. We found that even during present time, there is not much active development of new operating systems due to the immense knowledge required to design one. Therefore, in this project we try to develop an OS that performs basic operations such as displaying through VGA, taking input from keyboard and mouse and memory management.

Introduction

Operating System is the primary system software required for any bare computational system to run. The Operating System acts as a bridge between Hardware and the User. Operating System is the first program to start when the computer is turned on and all the other subsequent programs are launched by the OS. Depending upon the use, there are different types of OS such as distributed OS and real time OS. The use of OS is abundant in Embedded Systems, where miniature specific-purpose design is implemented rather than general-purpose design. Currently, most of the computerized system contains OS, making its availability everywhere. OS has two types of interface, Command Line Interface (CLI) and Graphical User Interface (GUI) making the use of OS simplistic and reliable. For the sake of basicity, this project is based on CLI. The lowest part of an OS is the kernel. Kernel is the core of an OS which handles and maintains all the processes, device drivers, I/O management and file management.

Objective

To implement Basic Kernel from scratch in order to understand the core of an Operating System.

Methodology

When we boot a computer, it needs to start up without the knowledge of an Operating System. It needs to search and load our Operating System to the memory. As a pre-OS environment, Motherboard contains Basic Input/output Software (BIOS), a collection of software routines that are initially loaded from a chip into memory and initialized when the computer is switched on. BIOS provides auto-detection and basic control of computer's essential devices such as screen, keyboard and hard disks.

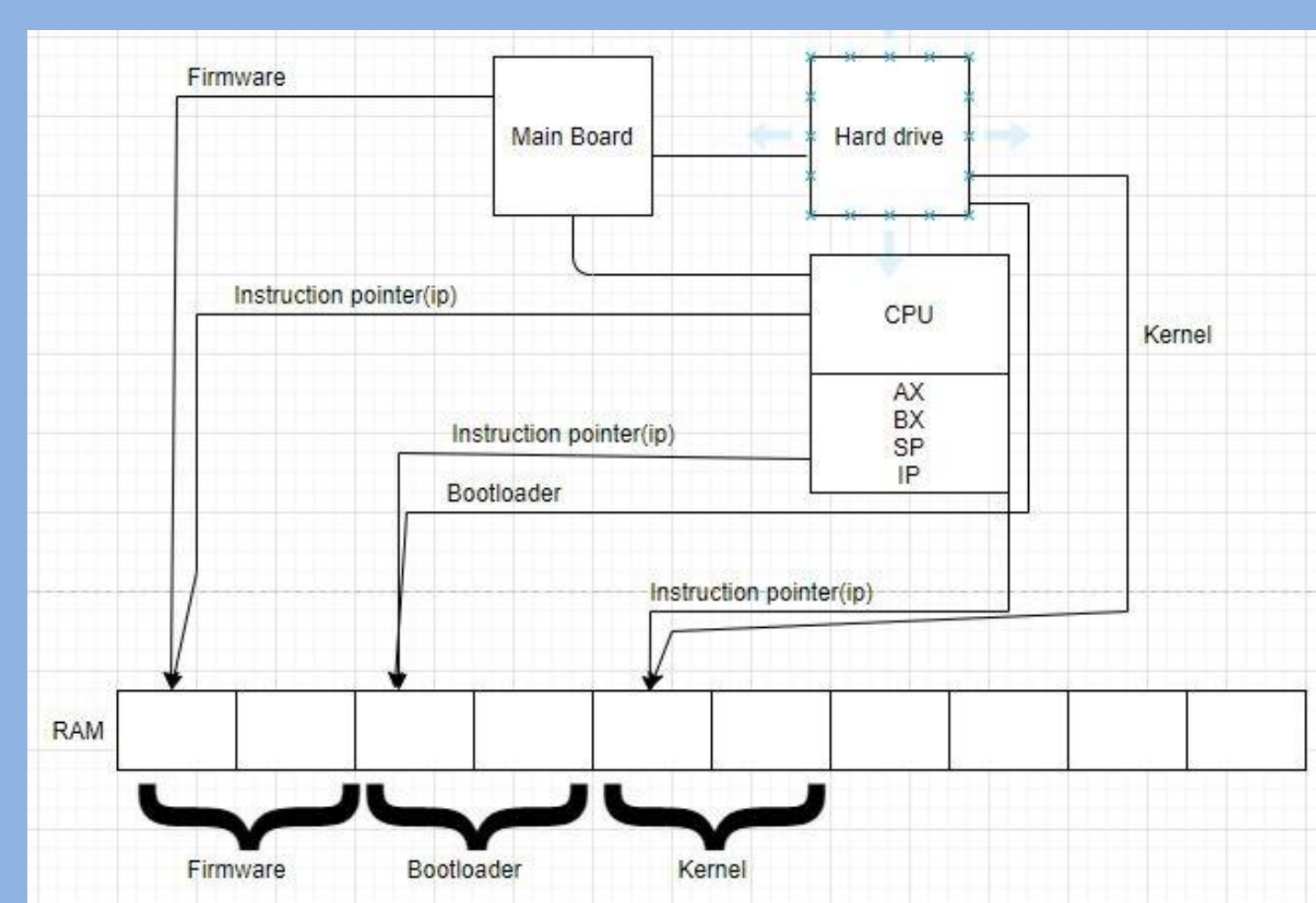


Figure: System flow during boot process

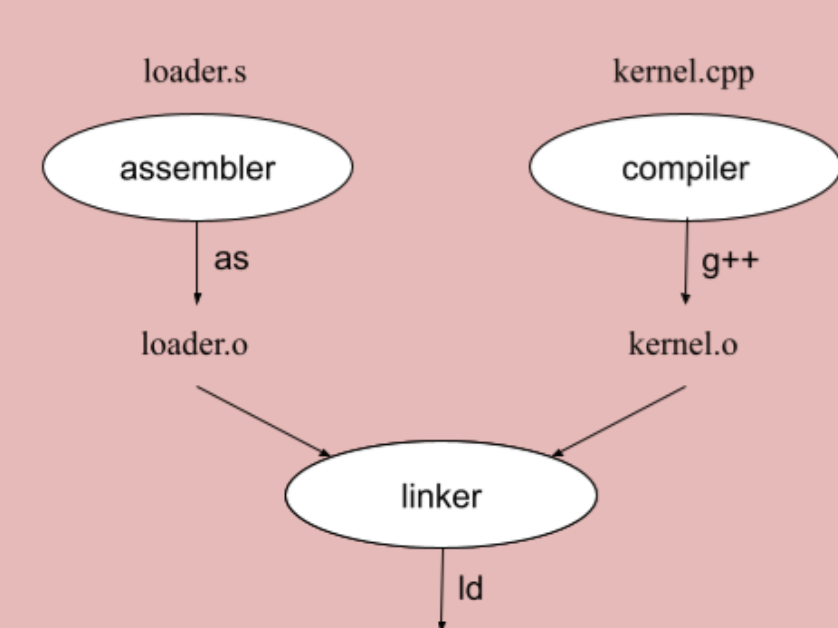


Figure: Process to creation of .bin file

GNU/Linux assembler, 'as' is used to assemble the assembly files whereas GNU/Linux compiler, 'g++' is used to compile cpp files. These produce two different types of object files, machine code output of an assembler or compiler. These two different object files need to be linked together which is done by GNU/Linux linker, 'ld' which gives us .bin file, binary executable file which executes the OS after being loaded into the main memory.

Global descriptor table (GDT) is the memory structure that describes the CPU attributes of the segment. Each descriptor in the table is 8-byte wide among which the first descriptor is always a null descriptor and cannot be used to access memory. Usually, this null descriptor is used to store the pointer to the GDT itself. The remaining descriptors in the GDT describe the attributes of the segment like base address of the segment, limit size of the segment, privilege level and the like.

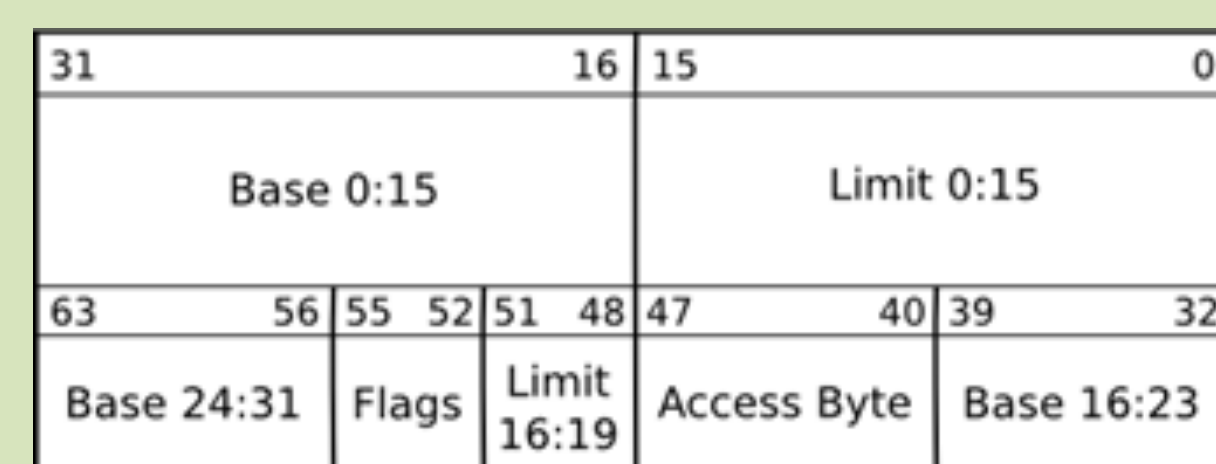


Figure: Structure of a single descriptor in the GDT

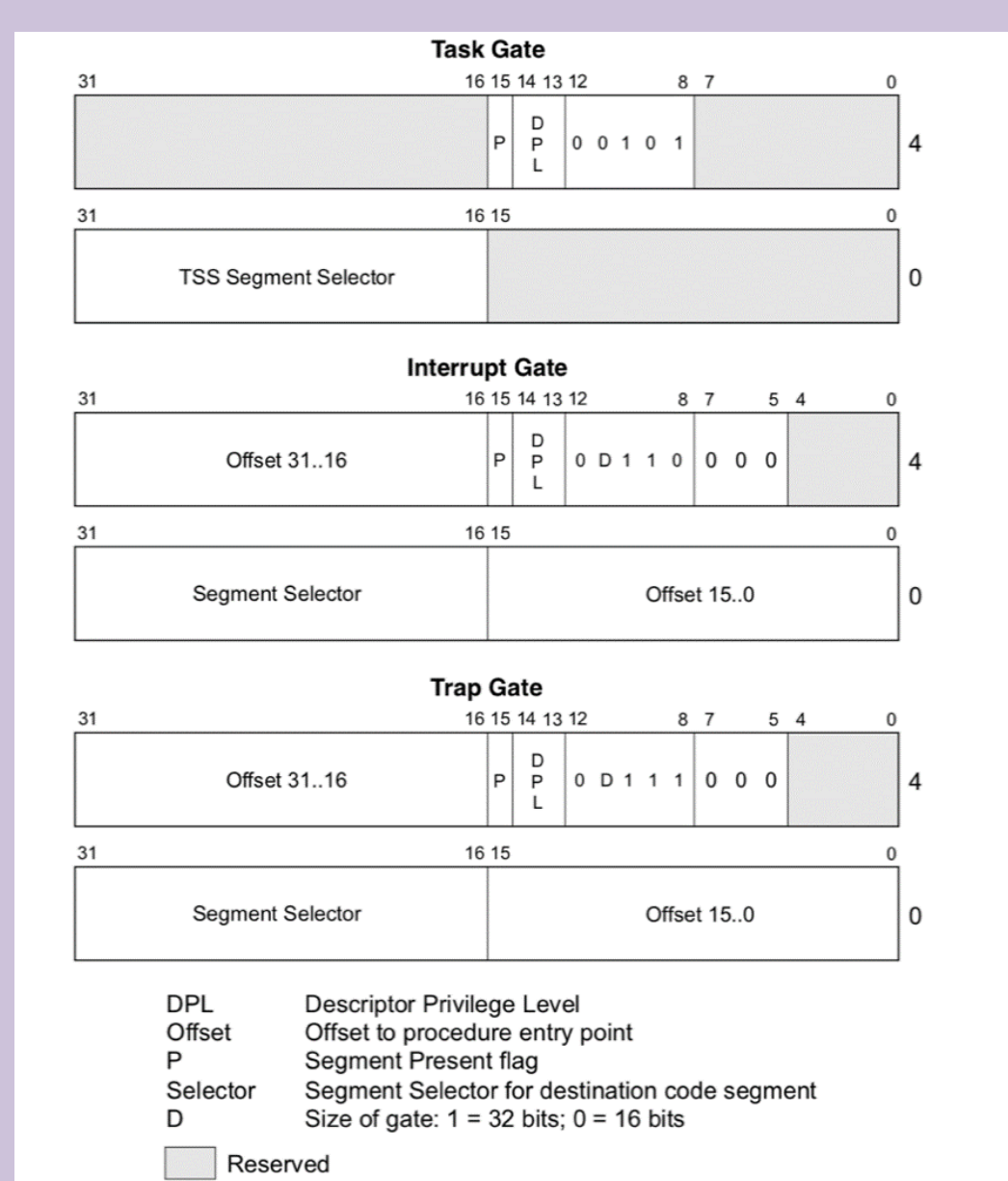


Figure: Structure of different gate descriptors in the IDT

When a specific interrupt arises, the CPU looks up for an entry for that specific interrupt from a table called Interrupt Descriptor Table (IDT). Protected mode uses Interrupt Descriptor Table (IDT) which is the counterpart of Real mode Interrupt Vector Table (IVT). For locating the address of Interrupt Service Routine (ISR). It is similar to the GDT in structure. The location of the IDT is stored in the IDTR register of the CPU, which can be loaded or stored using LIDT/SIDT instructions. Since, there are 256 different interrupts, so IDT should have 256 entries (gate descriptors), each corresponding to the specific interrupt. Each entry in IDT is 8-byte long and contains either of the three gate descriptors: task gate, interrupt gate or trap gate.

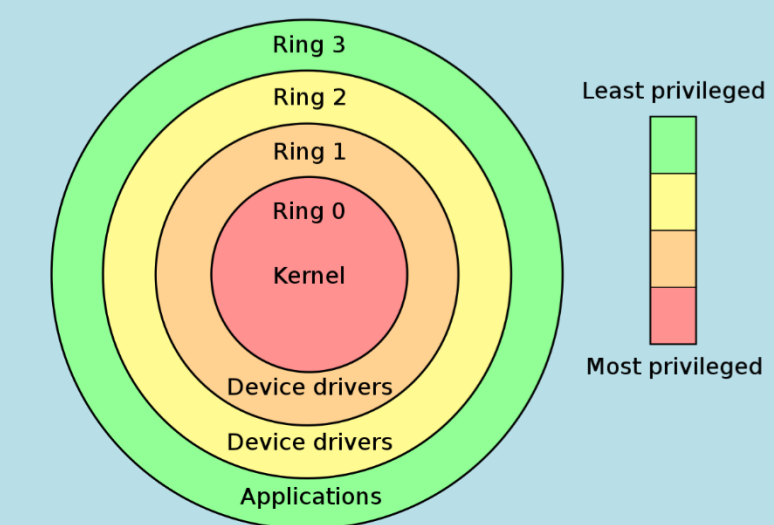


Figure: Protection ring

The different protection ring for privilege levels are:

- Ring 0: Accessible to the kernel, in kernel mode. This ring has direct access to the CPU and the system memory, so any instructions requiring the use of either will be executed here.
- Ring 1: It interacts with and controls hardware connected to the computer. Playing a song through speaker or displaying video on monitor are examples of instructions that would need to run in this ring.
- Ring 2: It is used for instructions that need to interact with system storage, loading or saving files (I/O).
- Ring 3: It is the least privileged ring, accessible to user processes that are running in user mode. This ring has no direct access to the CPU or memory and therefore has to pass any instructions involving these to ring 0.

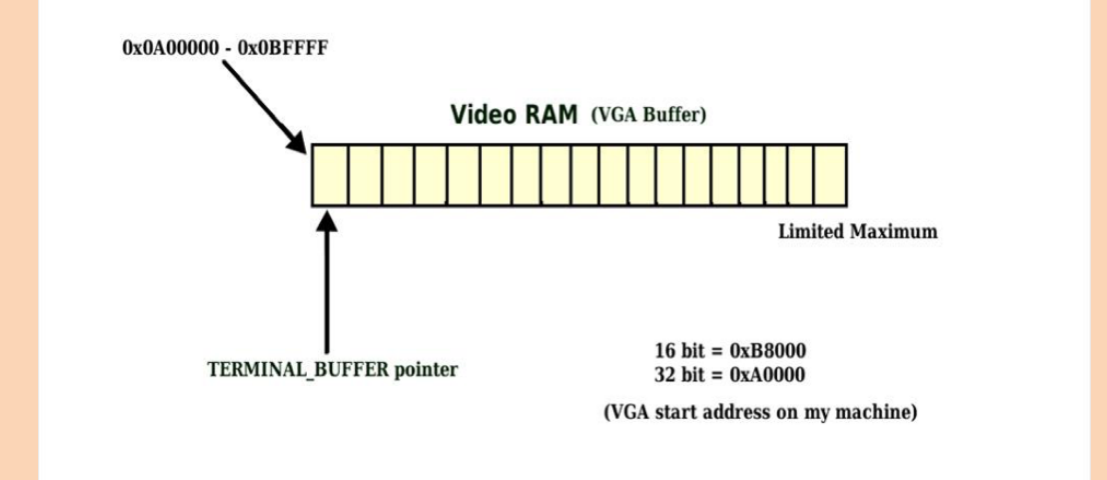


Figure: Structure of Memory Mapped VGA

In a 16-bit real mode system, VGA can be accessed using VGA Video BIOS Interface, a set of video interrupts (software interrupt 0x10). Because these are BIOS interrupts, they can only be used in real or v86 modes. For 32-bit protected mode, these BIOS interrupts cannot be used. Therefore, we have to manually create a service routine to access VGA through memory location.

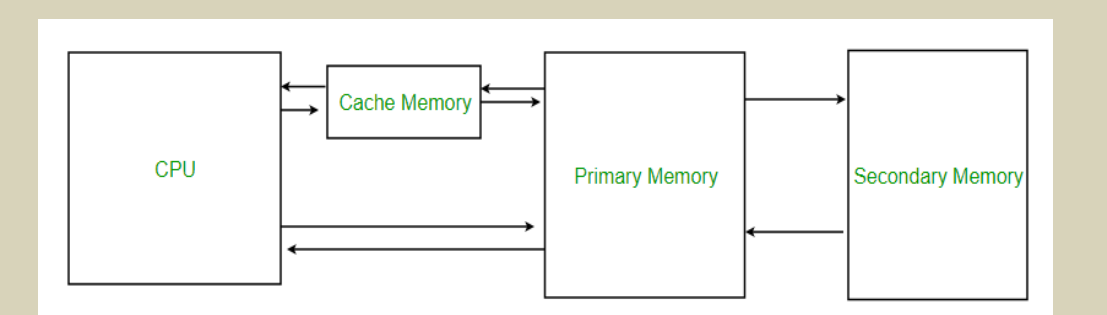


Figure: Basic Memory Hierarchy in the Computer Architecture

Results



Conclusion

The main purpose of selection of this project was to gain knowledge on how an operating system works on a bare hardware providing the user-friendly interface on the surface and management of operations on inside. On the timeline of project, we understood from loading of OS to the main memory to installing OS features such as system commands and arithmetic operations. In this minor project, we learnt and implemented the boot loading, GDT for segmentation of data and code segment, IDT for defining the Interrupts and VGA to display. During the process, we encountered numerous errors and problems due to complex data structure and algorithm. Such errors were tackled by the help of online OS Development community. At the final stage, we were able to write a basic operating system having basic features such as CUI, taking input from keyboard and mouse and performing operations by taking the pre-programmed commands.

References

- OSDev, OSDev, http://wiki.osdev.org/Main_Page [Accessed on Sep 10, 2020].
- Nick Blundell, Writing Operating System from Scratch, 2010
- Andrew S. Tanenbaum, 2008, Modern Operating Systems, Prentice Hall, Inc.
- Wikipedia, X86, <https://en.wikipedia.org/wiki/X86> [Accessed on Sep 20, 2020].
- Wikipedia, GNU GRUB, https://en.wikipedia.org/wiki/GNU_GRUB [Accessed on Sep 21, 2020].
- Wikipedia, VGA, https://en.wikipedia.org/wiki/Video_Graphics_Array [Accessed on Oct 11, 2020].
- Brokenthorn, VGA, <http://www.brokenthorn.com/Resources/OSDevVga.html> [Accessed on Oct 11, 2020].
- Wikipedia, Page (Computer Memory), [https://en.wikipedia.org/wiki/Page_\(computer_memory\)](https://en.wikipedia.org/wiki/Page_(computer_memory)) [Accessed on Oct 5, 2020].
- Erik Helin, Adam Renberg, The little book about OS development, <https://littleosbook.github.io/> [Accessed on Oct 4, 2020].
- IncludeHelp, <https://www.includehelp.com/operating-systems/> [Accessed on 6, 2020].